# LABORATORY WORK 7

## SQL Views and Roles

**Student:** Maman Naskhat
**Subject:** Database
**Topic:** SQL Views and Roles

## Objective

To understand and practice creating and managing SQL views (regular and materialized), implementing role-based access control, and learning how to secure and organize database objects effectively.

## Part 1: Database Setup

This laboratory work uses the same database tables from Laboratory Work 6:

- employees
- departments
- projects

No additional tables are required for this part.

# Part 2: Creating Basic Views

### Exercise 2.1: Simple View Creation

Create a view that shows employee names, salary, department name, and location.

```
CREATE VIEW employee_details AS
SELECT
    e.emp_name,
    e.salary,
    d.dept_name,
    d.location
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

**Test the view:**

```
SELECT * FROM employee_details;
```

**Answer:** The result contains 4 rows. Tom Brown does not appear because his department ID is NULL, and INNER JOIN only returns matching records.

### Exercise 2.2: View with Aggregation

```
CREATE VIEW dept_statistics AS
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS employee_count,
    ROUND(AVG(e.salary), 2) AS avg_salary,
    MAX(e.salary) AS max_salary,
    MIN(e.salary) AS min_salary
FROM departments d
LEFT JOIN employees e
ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
```

**Test the view:**

```
SELECT *
FROM dept_statistics
ORDER BY employee_count DESC;
```

This view displays department statistics, including departments without employees.

## Exercise 2.3: View with Multiple Joins

```
CREATE VIEW project_overview AS
SELECT
    p.project_name,
    p.budget,
    d.dept_name,
    d.location,
    COUNT(DISTINCT e.emp_id) AS team_size
FROM projects p
LEFT JOIN departments d
ON p.dept_id = d.dept_id
LEFT JOIN employees e
ON d.dept_id = e.dept_id
GROUP BY
    p.project_id,
    p.project_name,
    p.budget,
    d.dept_name,
    d.location;
```

This view shows project information, department details, and the number of employees in each project department.

## Exercise 2.4: View with Filtering

```
CREATE VIEW high_earners AS
SELECT
    e.emp_name,
    e.salary,
    d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id
WHERE e.salary > 55000;
```

**Question: What happens when you query this view? Can you see all high-earning employees?**

**Answer:** The view displays employees whose salary is greater than $55,000 and who have an assigned department. Employees without a department are not included because INNER JOIN is used.

---

# Part 3: Modifying and Managing Views

### Exercise 3.1: Replace a View

Modify employee_details to include a salary grade column.

```
CREATE OR REPLACE VIEW employee_details AS
SELECT
    e.emp_name,
    e.salary,
    d.dept_name,
    d.location,
    CASE
        WHEN e.salary > 60000 THEN 'High'
        WHEN e.salary > 50000 THEN 'Medium'
        ELSE 'Standard'
    END AS salary_grade
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

### Exercise 3.2: Rename a View

```
ALTER VIEW high_earners
RENAME TO top_performers;
```

**Verify:**

```
SELECT * FROM top_performers;
```

### Exercise 3.3: Drop a View

**Create temporary view:**

```
CREATE VIEW temp_view AS
SELECT *
FROM employees
WHERE salary < 50000;
```

**Drop the view:**

```
DROP VIEW temp_view;
```

---

# Part 4: Updatable Views

### Exercise 4.1: Create an Updatable View

```
CREATE VIEW employee_salaries AS
SELECT
    emp_id,
    emp_name,
    dept_id,
    salary
FROM employees;
```

## Exercise 4.2: Update Through a View

```
UPDATE employee_salaries
SET salary = 52000
WHERE emp_name = 'John Smith';
```

### Verify the update:

```
SELECT *
FROM employees
WHERE emp_name = 'John Smith';
```

**Answer:** Yes, the underlying employees table is updated because employee_salaries is a simple updatable view.

## Exercise 4.3: Insert Through a View

```
INSERT INTO employee_salaries
(emp_id, emp_name, dept_id, salary)
VALUES
(6, 'Alice Johnson', 102, 58000);
```

### Check the employees table:

```
SELECT * FROM employees;
```

**Answer:** Yes, the insert is successful. Alice Johnson is added to the underlying employees table through the view.

## Exercise 4.4: View with CHECK OPTION

### Create the view:

```
CREATE VIEW it_employees AS
SELECT
    emp_id,
    emp_name,
    dept_id,
    salary
FROM employees
WHERE dept_id = 101
WITH LOCAL CHECK OPTION;
```

### Try to insert an employee from another department:

```
INSERT INTO it_employees
(emp_id, emp_name, dept_id, salary)
VALUES
(7, 'Bob Wilson', 103, 60000);
```

**Answer:** PostgreSQL returns an error because the inserted employee does not satisfy the view condition (dept_id = 101). CHECK OPTION prevents inserting records that do not meet the view condition.

---

# Part 5: Materialized Views

## Exercise 5.1: Create a Materialized View

```
CREATE MATERIALIZED VIEW dept_summary_mv AS
WITH employee_stats AS (
    SELECT
        dept_id,
        COUNT(*) AS total_employees,
        SUM(salary) AS total_salaries
    FROM employees
    WHERE dept_id IS NOT NULL
    GROUP BY dept_id
),
project_stats AS (
    SELECT
        dept_id,
        COUNT(*) AS total_projects,
        SUM(budget) AS total_project_budget
    FROM projects
    WHERE dept_id IS NOT NULL
    GROUP BY dept_id
)
SELECT
    d.dept_id,
    d.dept_name,
    COALESCE(es.total_employees, 0) AS total_employees,
    COALESCE(es.total_salaries, 0) AS total_salaries,
    COALESCE(ps.total_projects, 0) AS total_projects,
    COALESCE(ps.total_project_budget, 0) AS total_project_budget
FROM departments d
LEFT JOIN employee_stats es
ON d.dept_id = es.dept_id
LEFT JOIN project_stats ps
ON d.dept_id = ps.dept_id
WITH DATA;
```

### Query the materialized view:

```
SELECT *
FROM dept_summary_mv
ORDER BY total_employees DESC;
```

## Exercise 5.2: Refresh Materialized View

### Insert a new employee:

```
INSERT INTO employees
(emp_id, emp_name, dept_id, salary)
VALUES
(8, 'Charlie Brown', 101, 54000);
```

**Query before refreshing:**

```
SELECT * FROM dept_summary_mv;
```

**Refresh the materialized view:**

```
REFRESH MATERIALIZED VIEW dept_summary_mv;
```

**Query after refreshing:**

```
SELECT * FROM dept_summary_mv;
```

**Answer:** Before refreshing, the materialized view contains old data. After refreshing, it includes the new employee and updated statistics.

## Exercise 5.3: Concurrent Refresh

**Create a unique index:**

```
CREATE UNIQUE INDEX idx_dept_summary
ON dept_summary_mv (dept_id);
```

**Refresh concurrently:**

```
REFRESH MATERIALIZED VIEW CONCURRENTLY dept_summary_mv;
```

**Answer:** CONCURRENTLY allows users to read the materialized view while it is being refreshed. It reduces blocking during the refresh operation.

## Exercise 5.4: Materialized View with NO DATA

```
CREATE MATERIALIZED VIEW project_stats_mv AS
SELECT
    p.project_name,
    p.budget,
    d.dept_name,
    COUNT(DISTINCT e.emp_id) AS employee_count
FROM projects p
LEFT JOIN departments d
ON p.dept_id = d.dept_id
LEFT JOIN employees e
ON d.dept_id = e.dept_id
GROUP BY
    p.project_id,
    p.project_name,
    p.budget,
    d.dept_name
WITH NO DATA;
```

**Try to query it:**

```
SELECT * FROM project_stats_mv;
```

**Answer:** PostgreSQL returns an error because the materialized view has not been populated.

**Fix:**

```
REFRESH MATERIALIZED VIEW project_stats_mv;
```

After refreshing, the view can be queried successfully.

---

# Part 6: Database Roles

## Exercise 6.1: Create Basic Roles

```
CREATE ROLE analyst NOLOGIN;

CREATE ROLE data_viewer
LOGIN PASSWORD 'viewer123';

CREATE USER report_user
WITH PASSWORD 'report456';
```

### View all roles:

```
SELECT rolname
FROM pg_roles
WHERE rolname NOT LIKE 'pg_%';
```

## Exercise 6.2: Role with Specific Attributes

```
CREATE ROLE db_creator
WITH LOGIN CREATEDB PASSWORD 'creator789';

CREATE ROLE user_manager
WITH LOGIN CREATEROLE PASSWORD 'manager101';

CREATE ROLE admin_user
WITH LOGIN SUPERUSER PASSWORD 'admin999';
```

## Exercise 6.3: Grant Privileges to Roles

### Grant SELECT on tables to analyst:

```
GRANT USAGE ON SCHEMA public TO analyst;

GRANT SELECT ON employees, departments, projects
TO analyst;
```

### Grant ALL PRIVILEGES on employee_details to data_viewer:

```
GRANT USAGE ON SCHEMA public TO data_viewer;

GRANT ALL PRIVILEGES ON employee_details
TO data_viewer;
```

### Grant SELECT and INSERT on employees to report_user:

```
GRANT USAGE ON SCHEMA public TO report_user;

GRANT SELECT, INSERT ON employees
```

```
TO report_user;
```

## Exercise 6.4: Create Group Roles

### 1. Create group roles:

```
CREATE ROLE hr_team NOLOGIN;
CREATE ROLE finance_team NOLOGIN;
CREATE ROLE it_team NOLOGIN;
```

### 2. Create individual users:

```
CREATE USER hr_user1 WITH PASSWORD 'hr001';
CREATE USER hr_user2 WITH PASSWORD 'hr002';
CREATE USER finance_user1 WITH PASSWORD 'fin001';
```

### 3. Assign users to hr_team:

```
GRANT hr_team TO hr_user1;
GRANT hr_team TO hr_user2;
```

### 4. Assign finance_user1 to finance_team:

```
GRANT finance_team TO finance_user1;
```

### 5. Grant SELECT and UPDATE on employees to hr_team:

```
GRANT USAGE ON SCHEMA public TO hr_team;

GRANT SELECT, UPDATE ON employees
TO hr_team;
```

### 6. Grant SELECT on dept_statistics to finance_team:

```
GRANT USAGE ON SCHEMA public TO finance_team;

GRANT SELECT ON dept_statistics
TO finance_team;
```

## Exercise 6.5: Revoke Privileges

### Revoke UPDATE privilege:

```
REVOKE UPDATE ON employees FROM hr_team;
```

### Revoke hr_team membership from hr_user2:

```
REVOKE hr_team FROM hr_user2;
```

### Revoke ALL PRIVILEGES on employee_details:

```
REVOKE ALL PRIVILEGES ON employee_details
FROM data_viewer;
```

## Exercise 6.6: Modify Role Attributes

**Add LOGIN and password to analyst:**

```
ALTER ROLE analyst
WITH LOGIN PASSWORD 'analyst123';
```

**Make user_manager a SUPERUSER:**

```
ALTER ROLE user_manager
WITH SUPERUSER;
```

**Remove password from analyst:**

```
ALTER ROLE analyst
PASSWORD NULL;
```

**Add connection limit of 5 to data_viewer:**

```
ALTER ROLE data_viewer
CONNECTION LIMIT 5;
```

---

# Part 7: Advanced Role Management

## Exercise 7.1: Role Hierarchies

### 1. Create parent role read_only:

```
CREATE ROLE read_only NOLOGIN;

GRANT USAGE ON SCHEMA public TO read_only;

GRANT SELECT ON ALL TABLES IN SCHEMA public
TO read_only;
```

### 2. Create child roles:

```
CREATE ROLE junior_analyst
WITH LOGIN PASSWORD 'junior123';

CREATE ROLE senior_analyst
WITH LOGIN PASSWORD 'senior123';
```

### 3. Grant read_only membership:

```
GRANT read_only TO junior_analyst;
GRANT read_only TO senior_analyst;
```

### 4. Grant additional privileges to senior_analyst:

```
GRANT INSERT, UPDATE ON employees
TO senior_analyst;
```

## Exercise 7.2: Object Ownership

### 1. Create project_manager:

```
CREATE ROLE project_manager
WITH LOGIN PASSWORD 'pm123';
```

### Grant CREATE permission on the schema:

```
GRANT CREATE ON SCHEMA public
TO project_manager;
```

### 2. Transfer ownership of dept_statistics:

```
ALTER VIEW dept_statistics
OWNER TO project_manager;
```

### 3. Transfer ownership of projects:

```
ALTER TABLE projects
OWNER TO project_manager;
```

### Check ownership:

```
SELECT tablename, tableowner
FROM pg_tables
WHERE schemaname = 'public';
```

## Exercise 7.3: Reassign and Drop Roles

### 1. Create temp_owner:

```
CREATE ROLE temp_owner
WITH LOGIN;
```

### 2. Create temp_table:

```
CREATE TABLE temp_table (
    id INT
);
```

### 3. Transfer ownership:

```
ALTER TABLE temp_table
OWNER TO temp_owner;
```

### 4. Reassign all objects to postgres:

```
REASSIGN OWNED BY temp_owner TO postgres;
```

### 5. Drop all remaining objects and privileges owned by temp_owner:

```
DROP OWNED BY temp_owner;
```

### 6. Drop the role:

```
DROP ROLE temp_owner;
```

## Exercise 7.4: Row-Level Security with Views

### 1. Create hr_employee_view:

```
CREATE VIEW hr_employee_view AS
SELECT *
FROM employees
WHERE dept_id = 102;
```

### 2. Grant SELECT to hr_team:

```
GRANT SELECT ON hr_employee_view
TO hr_team;
```

### 3. Create finance_employee_view:

```
CREATE VIEW finance_employee_view AS
SELECT
    emp_id,
    emp_name,
    salary
FROM employees;
```

### 4. Grant SELECT to finance_team:

```
GRANT SELECT ON finance_employee_view
TO finance_team;
```

---

# Part 8: Practical Scenarios

## Exercise 8.1: Department Dashboard View

```
CREATE VIEW dept_dashboard AS
WITH employee_stats AS (
    SELECT
        dept_id,
        COUNT(*) AS employee_count,
        AVG(salary) AS avg_salary
    FROM employees
    WHERE dept_id IS NOT NULL
    GROUP BY dept_id
),
project_stats AS (
    SELECT
        dept_id,
        COUNT(*) AS active_projects,
        SUM(budget) AS total_project_budget
    FROM projects
    WHERE dept_id IS NOT NULL
    GROUP BY dept_id
)
SELECT
    d.dept_name,
    d.location,
    COALESCE(es.employee_count, 0) AS employee_count,
    COALESCE(ROUND(es.avg_salary, 2), 0) AS avg_salary,
    COALESCE(ps.active_projects, 0) AS active_projects,
    COALESCE(ps.total_project_budget, 0) AS total_project_budget,
    COALESCE(
        ROUND(
```

```
            ps.total_project_budget /
            NULLIF(es.employee_count, 0),
            2
        ),
        0
    ) AS budget_per_employee
FROM departments d
LEFT JOIN employee_stats es
ON d.dept_id = es.dept_id
LEFT JOIN project_stats ps
ON d.dept_id = ps.dept_id;
```

**Test the view:**

```
SELECT * FROM dept_dashboard;
```

This view displays department information, employee statistics, project budgets, and budget per employee.

## Exercise 8.2: Audit View

### 1. Add created_date column:

```
ALTER TABLE projects
ADD COLUMN IF NOT EXISTS created_date TIMESTAMP
DEFAULT CURRENT_TIMESTAMP;
```

### 2. Create high_budget_projects view:

```
CREATE VIEW high_budget_projects AS
SELECT
    p.project_name,
    p.budget,
    d.dept_name,
    p.created_date,
    CASE
        WHEN p.budget > 150000
            THEN 'Critical Review Required'
        WHEN p.budget > 100000
            THEN 'Management Approval Needed'
        ELSE 'Standard Process'
    END AS approval_status
FROM projects p
LEFT JOIN departments d
ON p.dept_id = d.dept_id
WHERE p.budget > 75000;
```

**Test the view:**

```
SELECT * FROM high_budget_projects;
```

This view displays projects with budgets greater than $75,000 and assigns an approval status based on their budget.

## Exercise 8.3: Create Access Control System

### Level 1 - Viewer Role

```
CREATE ROLE viewer_role NOLOGIN;

GRANT USAGE ON SCHEMA public
TO viewer_role;

GRANT SELECT ON ALL TABLES IN SCHEMA public
TO viewer_role;
```

### Level 2 - Entry Role

```
CREATE ROLE entry_role NOLOGIN;

GRANT viewer_role TO entry_role;

GRANT INSERT ON employees, projects
TO entry_role;
```

### Level 3 - Analyst Role

```
CREATE ROLE analyst_role NOLOGIN;

GRANT entry_role TO analyst_role;

GRANT UPDATE ON employees, projects
TO analyst_role;
```

### Level 4 - Manager Role

```
CREATE ROLE manager_role NOLOGIN;

GRANT analyst_role TO manager_role;

GRANT DELETE ON employees, projects
TO manager_role;
```

### Create Users:

```
CREATE USER alice WITH PASSWORD 'alice123';
CREATE USER bob WITH PASSWORD 'bob123';
CREATE USER charlie WITH PASSWORD 'charlie123';
```

### Assign users to roles:

```
GRANT viewer_role TO alice;

GRANT analyst_role TO bob;

GRANT manager_role TO charlie;
```

### Verify role membership:

```
SELECT
    r.rolname AS role_name,
    m.rolname AS member_name
FROM pg_auth_members am
JOIN pg_roles r
ON am.roleid = r.oid
JOIN pg_roles m
ON am.member = m.oid
ORDER BY role_name;
```